UNIVERSITAT POLITÈCNICA DE VALÈNCIA

etsinf

Tema 2

Representación del Conocimiento basada en la Lógica



INDICE

1. Introducción.
2. Lógica proposicional y lógica de predicados de primer orden.
3. Programación lógica.
   1. Claúsulas de Horn
   2. Unificación
   3. Negación por fallo
   4. Resolución SLD
4. Propiedades de la lógica como formalismo de representación del conocimiento.
5. Ejercicios.



Métodos de Representación Basados en la Lógica

Estructuras de representación:

Representación de los hechos del mundo real mediante declaraciones escritas como fórmulas bien formadas (fbf), o estructuras sintácticamente correctas del lenguaje.

Mecanismos de interpretación:

Obtención de nuevo conocimiento a partir del antiguo (reglas que permitan obtener nuevas fbf a partir de las existentes).

LÓGICA := SINTAXIS + SEMÁNTICA



Métodos de Representación Basados en la Lógica

- Sistemas lógicos:
  - Lógica proposicional
  - Lógica de primer orden
    - Cuantificación sobre individuos
  - Lógicas de orden superior
    - Por ejemplo, la lógica de segundo orden cuantifica sobre conjuntos
  - Lógicas modales
    - La lógica modal clásica añade expresiones de modalidad (es necesario que, es posible que)
    - También existen otras lógicas modales (lógicas temporales)



La lógica en IA

- Se refiere al uso aplicado y computacional de la lógica para **representar y razonar sobre el conocimiento** en problemas.
- Utiliza conceptos de la lógica formal pero orientados a la representación del conocimiento y razonamiento para la resolución de problemas

1. **Representación del conocimiento** con lógica de predicados (lógica de primer orden – FOL (first order logic))
2. **Razonamiento automático** para resolver problema: se puede implementar con diferentes modelos como programación lógica, sistemas basados en reglas u otros tipos de motores de inferencia.



La lógica en IA

¿Cuáles son las diferencias entre la lógica como disciplina formal y la lógica como la lógica en IA?

	Lógica formal	Lógica en IA
Enfoque	Teórico	Aplicado (razonamiento automático)
Razonamiento	Deductivo	Deductivo (incertidumbre, incompletitud, inconsistencia)
Objetivo principal	Estudiar principios del razonamiento	Representación del conocimiento y razonamiento automático
Lenguaje	Formal y simbólico	Adaptado a sistemas computacionales
Uso	Demostraciones formales	Sistemas expertos, razonamiento

Razonamiento automático basado en la programación lógica



¿Por qué representar conocimiento con lógica?

- Claridad y precisión: Representación formal, sin ambigüedades. Cada enunciado tiene un significado lógico bien definido.
- Razonamiento automático: Permite inferir nuevo conocimiento a partir de reglas. Ejemplo:

  $\forall x\,(\text{Humano}(x) \rightarrow \text{Mortal}(x))$

  $\text{Humano}(\text{Sócrates})$

  $\vdash \text{Mortal}(\text{Sócrates})$
- Modularidad y generalización: Reglas universales reutilizables. Ejemplo:

  $\forall x\,(\text{Estudiante}(x) \rightarrow \text{TieneCorreo}(x))$
- Fundamento de muchos sistemas inteligentes como agentes, sistemas expertos, verificación formal, bases de datos lógicas…
- Base teórica sólida que es estudiable formalmente: correctitud, completitud, decidibilidad.

La lógica es una herramienta esencial para representar conocimiento de forma precisa y para razonar automáticamente con él.



Programación lógica

- Paradigma de programación que utiliza la lógica formal para representar el conocimiento y razonar sobre él
- Herramienta para construir sistemas de IA que permite razonar de manera inteligente a partir de un conocimiento previamente definido

Trabajaremos con un subconjunto de funcionalidades de la programación lógica para representar el conocimiento y aplicar razonamiento automático



Lógica Proposicional

- Lenguaje de representación que nos permite expresar y razonar con sentencias que son verdaderas o falsas .

"El tren llega a las 7", "Hay taxis en la estación"

- **Lenguaje:**
  - Alfabeto proposicional:
    - Variables proposicionales: P, Q, R…
    - Conectivas lógicas: $\neg$ (negación), $\wedge$ (conjunción), $\vee$ (disyunción), $\rightarrow$ (condicional), $\leftrightarrow$ (bicondicional).
    - Símbolos auxiliares: (,)
  - Fórmulas proposicionales:
    - Las variables proposicionales son fórmulas
    - Si F y G son fórmulas, entonces también lo son: $\neg F$, $(F \wedge G)$, $(F \vee G)$, $(F \rightarrow G)$ y $(F \leftrightarrow G)$

Ejemplo: G = "La aorta es una arteria grande"

D = "La aorta tiene un diámetro de 2.5 cm"

G^D "La aorta es una arteria grande y la aorta tiene un diámetro de 2.5 cm"



Lógica Proposicional

Semántica: establece cuándo una fórmula es verdadera.

- **Interpretación**: asigna un valor de verdad (V o F) a cada proposición.
- **Valor de verdad**: toda fórmula es verdadera o falsa bajo una interpretación.
- Tablas de verdad:

i	$\neg i$
1	0
0	1

i	j	$i \wedge j$	$i \vee j$	$i \rightarrow j$	$i \leftrightarrow j$
1	1	1	1	1	1
1	0	0	1	0	0
0	1	0	1	1	0
0	0	0	0	1	1



Semántica

- **Modelo**: interpretación en la que una fórmula resulta verdadera.

Ejemplo: Dada $F = (p \vee q) \wedge (\neg q \vee r)$

Interpretación: I(p)=1, I(q)=0, I(r)=1

$$\begin{array}{ccccccc} (p & \vee & q) & \wedge & (\neg q & \vee & r) \\ (1 & \vee & 0) & \wedge & (\neg 0 & \vee & 1) \\ & 1 & & \wedge & (1 & \vee & 1) \\ & 1 & & \wedge & & 1 & \\ & & & 1 & & & \end{array}$$

⇩

Es un modelo

Ejercicio:

¿La interpretación I(p)=0, I(q)=0, I(r)=1 es un modelo?



Semántica

Consecuencia

$$\Gamma \models \varphi$$

φ es verdadera en todos los modelos de Γ.

Por ejemplo: $P, P \rightarrow Q \models Q$ El único modelo es P=1 y Q=1

P	Q	P→Q	¿Modelo?	Q
1	1	1	Si	1
1	0	0	No	0
0	1	1	No	1
0	0	1	No	0

Inferencia

$$\Gamma \vdash \varphi$$

Existe un procedimiento que permite obtener φ.



Ejercicio

Sean:

- L: Laura va al cine
- M: Marcos va al cine

Sabemos que (F) Laura o Marcos van al cine (o los dos), pero no sabemos cuál(es).

1. Representa la fórmula F
2. Determina si las siguientes fórmulas son consecuencia lógica de F
   1. L
   2. L v M
   3. ¬L → M



Lógica de Primer Orden

- **Limitaciones de la lógica proposicional**:

Juan es estudiante	JUANESTUDIANTE	⟹	Estudiante(JUAN)
María es estudiante	MARIAESTUDIANTE		Estudiante(MARIA)
Todos los hombres son mortales	HOMBREMORTAL		∀x: Hombre(x) → Mortal(x)

- **Lenguaje:**
  - Símbolos de predicado (P,Q,...): representan una relación en un dominio
    $P(t_1, ..., t_n)$, $n \neq 0$ (ti: términos)
  - Variables (x, y, z)
  - Constantes (a, b, c)
  - Símbolos de función (f, g, h): $f(t_1, ..., t_n)$, $n \neq 0$ ($f:D^n \rightarrow D$).
  - Conectivas lógicas: ¬ ^ v → ↔
  - Símbolo de igualdad: =
  - Cuantificador universal ∀, Cuantificador existencial ∃.
    ∀ **x:** "para todo x" ∃**x:** "existe un x"

Ejemplo:
∃ x: Planeta(x) --> Gira(x, Sol)



Lógica de Primer Orden

Una fórmula atómica, o átomo, es una expresión de la forma $P(t_1, \ldots, t_n)$ donde P es un símbolo de predicado n-ario, $n \neq o$, y $t_1, \ldots, t_n$ son términos.

Sintaxis de la lógica de primer orden.

Una fórmula bien formada (fbf) se define:

- Un átomo es una fbf.
- Sea F una fbf con un conjunto asociado de variables libres. Entonces (F) y ¬ F son fbf.
- Sean F y G fbf. Entonces FvG, F^G, F–>G y F<->G son fbf.
- Si F es una fbf y x es una variable de F, entonces (∀ x: F) y (∃ x: F) son fbf.
- Nada más es una fbf.



Expresividad de la Lógica de Primer Orden

- *La Tierra es un planeta:*
  $\text{planeta}(\text{Tierra})$
- *La Luna no es un planeta:*
  $\neg\,\text{planeta}(\text{Luna})$
- *La Luna es un satélite:*
  $\text{satélite}(\text{Luna})$
- *La Tierra gira alrededor del Sol:*
  $\text{gira}(\text{Tierra}, \text{Sol})$
- *Todo planeta es un satélite:*
  $\forall x\ [\text{planeta}(x) \rightarrow \text{satélite}(x)]$
- *Todo planeta gira alrededor del Sol:*
  $\forall x\ [\text{planeta}(x) \rightarrow \text{gira}(x, \text{Sol})]$
- *Algún planeta gira alrededor de la Luna:*
  $\exists x\ [\text{planeta}(x) \wedge \text{gira}(x, \text{Luna})]$
- *Algún planeta no tiene satélites:*
  $\exists x\ [\text{planeta}(x) \wedge \neg\exists y\ \text{satélite}(y, x)]$
- *Sólo los planetas tienen satélites:*
  $\forall x\ [\exists y\ \text{satélite}(y, x) \rightarrow \text{planeta}(x)]$
- *Todo satélite es satélite de algún planeta:*
  $\forall x\ [\text{satélite}(x) \rightarrow \exists y\ (\text{planeta}(y) \wedge \text{satélite}(x, y))]$
- *La Luna no gira alrededor de dos planetas diferentes:*
  $\neg\exists x\ \exists y\ [\ \text{planeta}(x) \wedge \text{planeta}(y) \wedge \text{gira}(\text{Luna}, x) \wedge \text{gira}(\text{Luna}, y) \wedge x \neq y]$
- *Hay exactamente dos planetas:*
  $\exists x\ \exists y\ [\ \text{planeta}(x) \wedge \text{planeta}(y) \wedge x \neq y \wedge \forall z\ [\text{planeta}(z) \rightarrow z = x \vee z = y]]$



Razonamiento en la Lógica: sistemas de demostración

1. Métodos Sintácticos (Usan reglas formales de inferencia para construir demostraciones)

Método	Idea principal	Características clave
Deducción Natural	Aplicar reglas de inferencia directamente (ej. modus ponens, introducción/eliminación de conectivos)	Intuitivo, cercano al razonamiento humano
Sistema de Hilbert	Axiomas + reglas (principalmente modus ponens)	Muy formal, pocas reglas, demostraciones largas
Resolución	Refutación: demostrar que $\Gamma \cup \{\neg\varphi\}$ (en forma normal conjuntiva, CNF) es inconsistente	Muy usada en lógica proposicional y automatización

2. Métodos Semánticos (Se basan en el análisis de modelos que satisfacen las fórmulas)

Método	Idea principal	Características clave
Tablas de Verdad	Comprobar validez en todas las combinaciones posibles	Solo para lógica proposicional
Modelos e Interpretaciones	Verificar que todo modelo de Γ satisface φ	Central en lógica de predicados
Contramodelos	Encontrar un modelo donde Γ sea verdadera y φ falsa	Útil para demostrar no-derivabilidad



Programación lógica: cláusulas de Horn

Una cláusula de Horn es una fórmula lógica que contiene como máximo un literal positivo (fórmula atómica positiva).

Una cláusula de Horn es disyunción de literales donde un solo término es positivo y los demás son negativos. Se denominan *definite Horn clauses*.

$$\neg p1 \vee \neg p2 \vee \ \ldots \vee \ \neg pn \vee q$$

Podemos representar una cláusula de Horn en forma de implicación (aplicando equivalencia condicional-disyunción):

$$p1 \wedge p2 \wedge \ \ldots \wedge \ pn \ \rightarrow \ q$$

regla lógica

fórmulas atómicas (cuerpo de la regla)

fórmula atómica (cabeza de la regla)



Programación lógica: cláusulas de Horn

$$p1 \wedge p2 \wedge \ldots \wedge pn \rightarrow q$$

Las variables de una regla lógica se interpretan universalmente cuantificadas (para todo objeto del dominio que satisfaga/unifique con las fórmulas del cuerpo de la regla)

- Las cláusulas de Horn son fundamentales en los lenguajes de programación lógica (por ejemplo, Prolog)
- Se usan para codificar bases de conocimiento y posteriormente poder razonar sobre ellas
- Un **hecho** (sentencia o fórmula TRUE) puede verse como una **regla sin cuerpo**



Ejemplo

Base de Conocimiento:

H1. padre (jose, maría)
H2. madre (elvira, jose)
R1. padre (X,Y) $\wedge$ madre (Z,X) $\rightarrow$ abuela (Z,Y)

Objetivos

padre (jose, maría)?
madre (elvira, maría)?
abuela (elvira, jose)?
abuela (elvira, maría)?



Programación lógica: unificación

Unificación: proceso algorítmico que busca hacer dos expresiones o cláusulas idénticas mediante la aplicación de sustituciones a las variables involucradas

Objetivo/sub-objetivo	Cabeza de la regla	Unificador
padre (jose, maría)	padre (jose, maría)	TRUE
	padre (jose, X)	X=maría
	madre (jose, maría)	FALSE
	padre (X, maría)	X=jose
	padre (X,Y)	X=jose, Y=maría
padre (jose, X)	padre (jose, Y)	Y=X
	padre (Z, Y)	Z=jose, Y=X
padre (X, Y)	padre (Z, W)	Z=X, W=Y



Programación lógica: negación por fallo

En lógica clásica, ¬p se interpreta semánticamente como: ¬p es verdadero si p es falso y viceversa.

En programación lógica se utiliza la **negación por fallo**, un mecanismo inferencial que interpreta una consulta negada como verdadera solo si no se puede encontrar ninguna prueba de la consulta original bajo la suposición de mundo cerrado (es decir, que todo lo que es cierto en el dominio se ha especificado).

Negación por fallo: ¬p es cierto si no se puede probar p



Programación lógica: negación por fallo

Base de Conocimiento:

H1. padre (jose, maría)
H2. madre (elvira, jose)
R1. padre (X,Y) $\wedge$ madre (Z,X) $\rightarrow$ abuela (Z,Y)

Objetivo: ¬padre (jose, elvira)?

padre (jose, elvira)? $\Longrightarrow$ No se puede probar porque no unifica con H1 ni H2 ni la cabeza de la regla R1

Por tanto, ¬padre (jose, elvira) es TRUE, es decir, jose no es el padre de elvira



Deducción en una BC

Dada una Base de Conocimiento (**BC**) formada por hechos y reglas, y una fórmula objetivo **P**, queremos responder si **P** puede demostrarse, inferirse u obtenerse de **BC**.

Se trata de implementar un proceso de razonamiento que busca una secuencia de aplicaciones válidas de la regla de Modus Ponens que obtenga **P** como fórmula final a partir de **BC**

La búsqueda de esta secuencia de aplicaciones válidas puede realizarse:

- hacia **atrás**, partiendo de **P** (se parte de la hipótesis **P** hasta llegar a los hechos de **BC**) – razonamiento dirigido por los objetivos
- hacia **adelante**, partiendo de **BC** (se parte de los hechos de **BC** hasta llegar a la conclusión **P**) – razonamiento dirigido por los datos



Programación lógica: resolución SLD

La regla de resolución es un principio fundamental en programación lógica que permite derivar nuevas conclusiones a partir de hechos y reglas existentes.

La resolución SLD (**S**elective **L**inear **D**efinite clause resolution) es un refinamiento específico de la regla de resolución que se aplica a cláusulas de Horn.

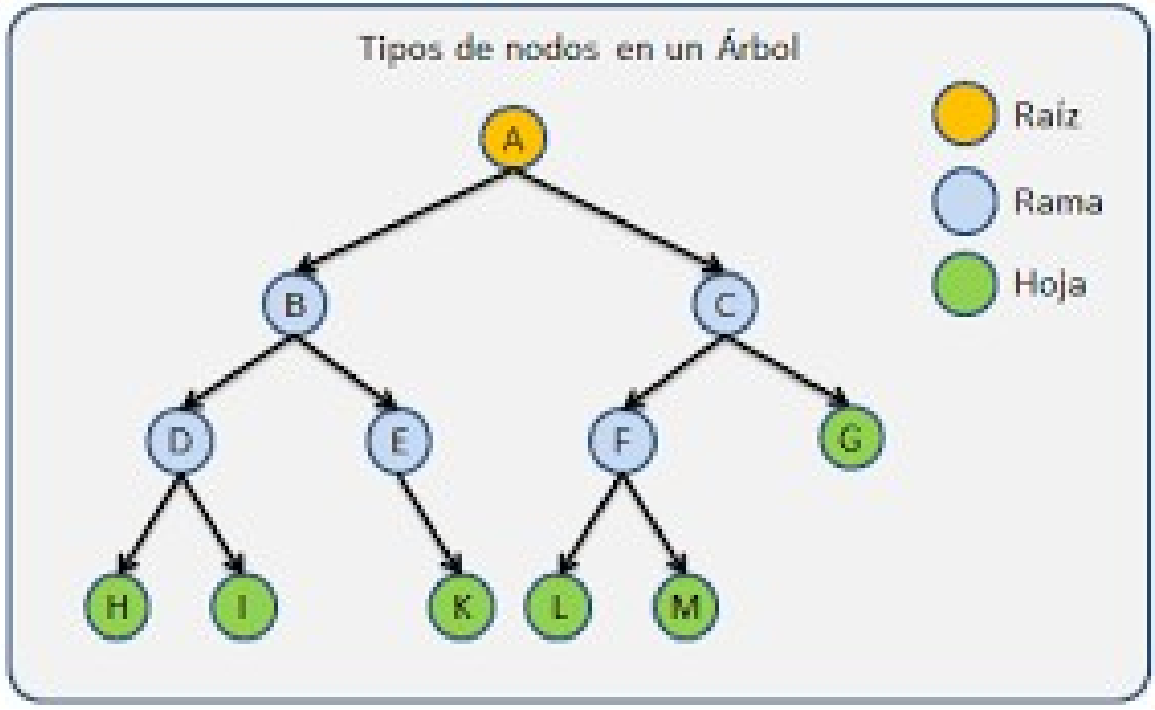
Es el método de inferencia principal en programación lógica (lenguajes como Prolog).

Es una forma de **razonamiento orientado a objetivos**, donde se busca probar si un objetivo **P** se deriva lógicamente de una **BC** (emplearemos razonamiento hacia atrás).



Programación lógica: resolución SLD

El algoritmo de resolución SLD realiza una búsqueda para satisfacer **P** en **BC** que se representa en un ***árbol***



- estructura de datos jerárquica
- organiza los datos en elementos llamados nodos y conectados por ramas



Programación lógica: resolución SLD

- Un árbol SLD representa el proceso de búsqueda.
- Los **nodos del árbol representan los objetivos pendientes por satisfacer**.
- El nodo raíz representa el objetivo inicial.
- Paso de resolución para un objetivo **P** (nodo): encontrar una unificación para **P** y generar nuevas cláusulas/objetivos (rama del nodo):
  - encontrar una cabeza de regla que unifique con P
  - el cuerpo de la regla son las nuevas cláusulas/objetivos a satisfacer
- Una rama termina en una **derivación de éxito** cuando se llega a la cláusula vacía (TRUE)
- Una rama termina en una **derivación de fallo** si no se pueden aplicar más pasos de resolución.

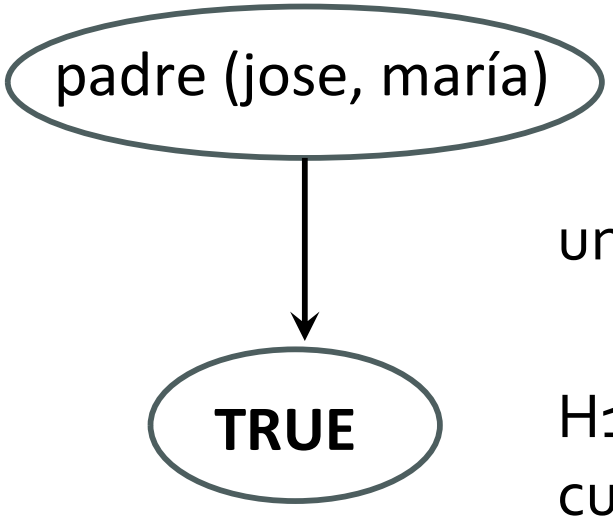


Programación lógica: resolución SLD

Base de Conocimiento (BC):

H1. padre (jose, maría)
H2. madre (elvira, jose)
R1. padre (X,Y) $\wedge$ madre (Z,X) $\rightarrow$ abuela (Z,Y)

Objetivo: padre (jose, maría)



buscamos regla que unifique con el objetivo

unificación con H1

el cuerpo de la regla con la que unifica son las nuevas cláusulas a satisfacer

H1 es un hecho (regla sin cuerpo) y por tanto la nueva cláusula es la cláusula VACÍA.

ÉXITO



Programación lógica: resolución SLD

Base de Conocimiento (BC):

H1. padre (jose, maría)
H2. madre (elvira, jose)
R1. padre (X,Y) $\wedge$ madre (Z,X) $\rightarrow$ abuela (Z,Y)

Objetivo: madre (elvira, maría)

madre (elvira, maría)

buscamos regla que unifique con el objetivo

FALLO → FALSE

No encontramos regla que unifique con el objetivo. Por tanto, no hay pasos de resolución que resuelvan el objetivo.



Programación lógica: resolución SLD

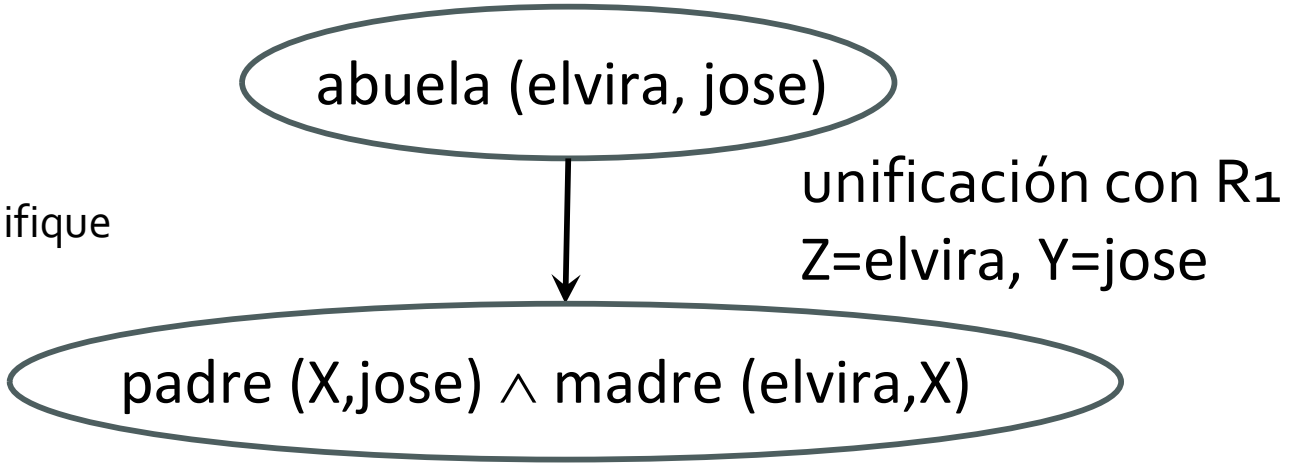
Base de Conocimiento (BC):

H1. padre (jose, maría)

H2. madre (elvira, jose)

R1. padre (X,Y) $\wedge$ madre (Z,X) $\rightarrow$ abuela (Z,Y)

Objetivo: abuela (elvira, jose)



buscamos regla que unifique con el objetivo

comenzamos por la cláusula más a la izquierda

no encontramos unificación para padre (X,jose)

FALLO → FALSE



Programación lógica: resolución SLD

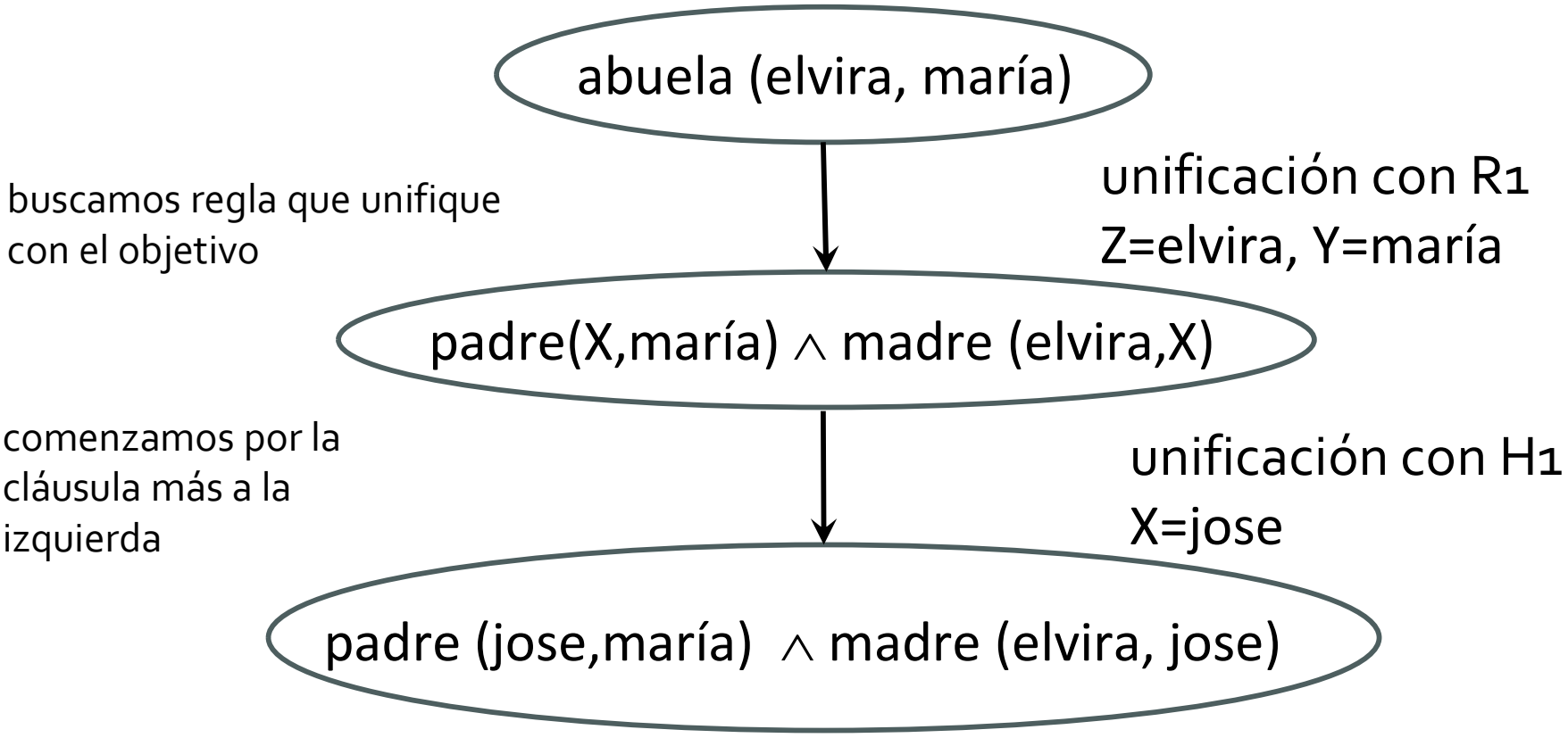
Base de Conocimiento (BC):

H1. padre (jose, maría)

H2. madre (elvira, jose)

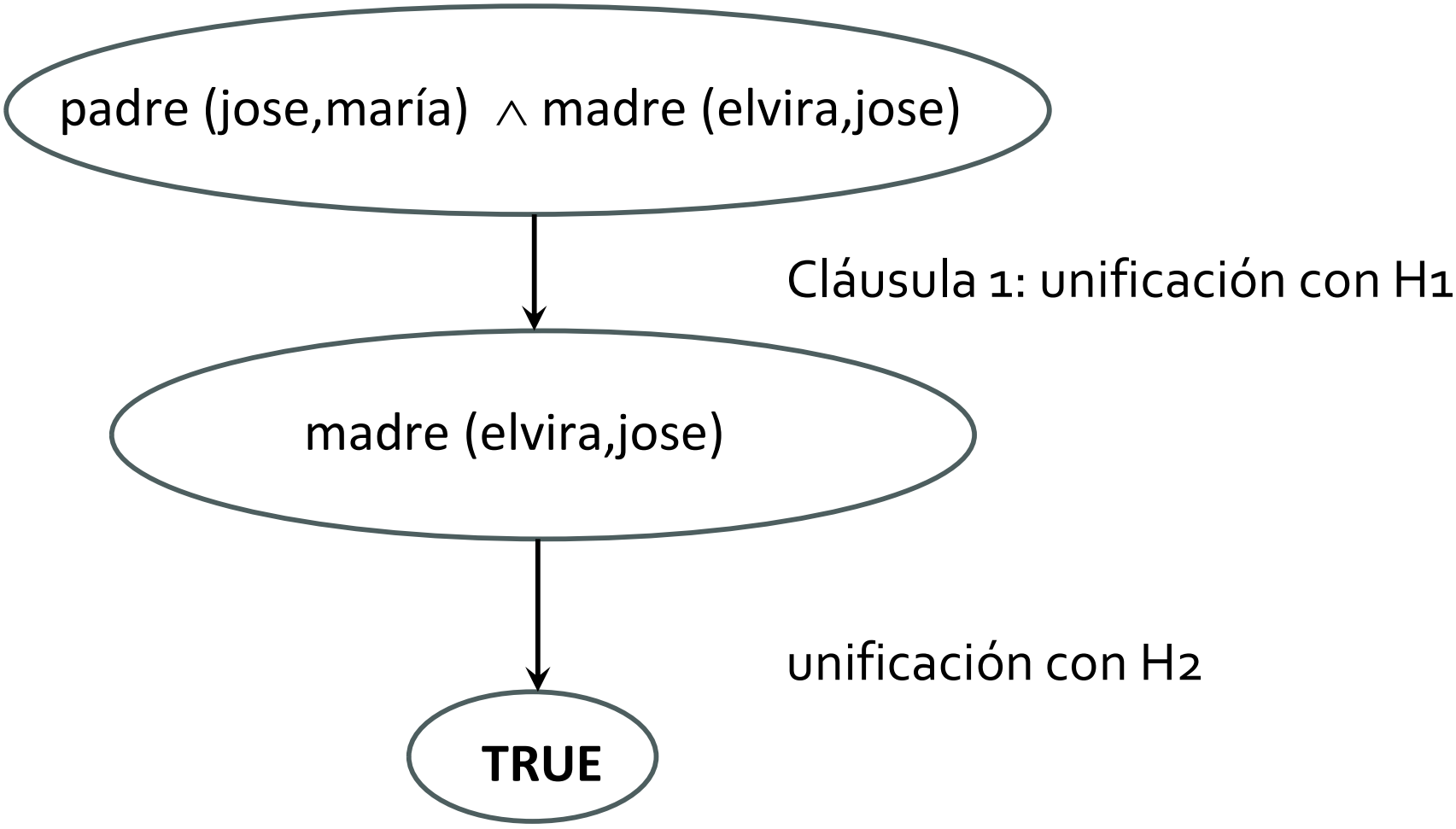
R1. padre (X,Y) $\wedge$ madre (Z,X) $\rightarrow$ abuela (Z,Y)

Objetivo: abuela (elvira, maría)





Programación lógica: resolución SLD



ÉXITO



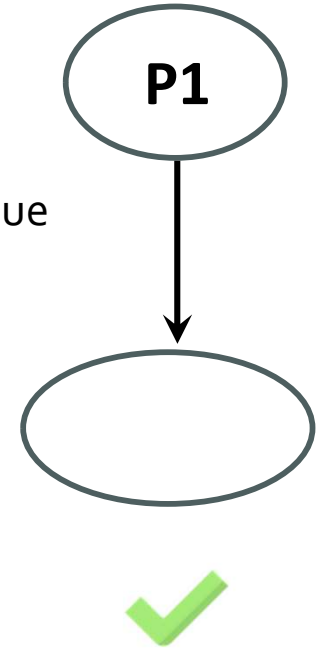
Programación lógica: resolución SLD

Ejemplo 2

Base de Conocimiento (BC):

H1. P1 $(\rightarrow$ P1)
H2. P2 $(\rightarrow$ P2)
H3. Q1 $(\rightarrow$ Q1)
R1. Q1 $\wedge$ Q2 $\rightarrow$ P3
R2. P2 $\rightarrow$ Q2
R3. P1 $\wedge$ Q4 $\rightarrow$ Q3
R4. P1 $\wedge$ P3 $\rightarrow$ Q3

Objetivo P1



unificación con H1

buscamos regla que unifique con el objetivo

el cuerpo de la regla con la que unifica son las nuevas cláusulas a satisfacer

H1 es un hecho (regla sin cuerpo) y, por tanto, la nueva cláusula es la cláusula VACÍA.



Programación lógica: resolución SLD

Base de Conocimiento (BC):

H1. P1 ($\rightarrow$ P1)
H2. P2 ($\rightarrow$ P2)
H3. Q1 ($\rightarrow$ Q1)
R1. Q1 $\wedge$ Q2 $\rightarrow$ P3
R2. P2 $\rightarrow$ Q2
R3. P1 $\wedge$ Q4 $\rightarrow$ Q3
R4. P1 $\wedge$ P3 $\rightarrow$ Q3

buscamos regla que unifique con el objetivo

Objetivo Q4

Q4

No encontramos regla cuya cabeza unifique con el objetivo. Por tanto, no hay pasos de resolución que resuelvan el objetivo.

FALLO



Programación lógica: resolución SLD

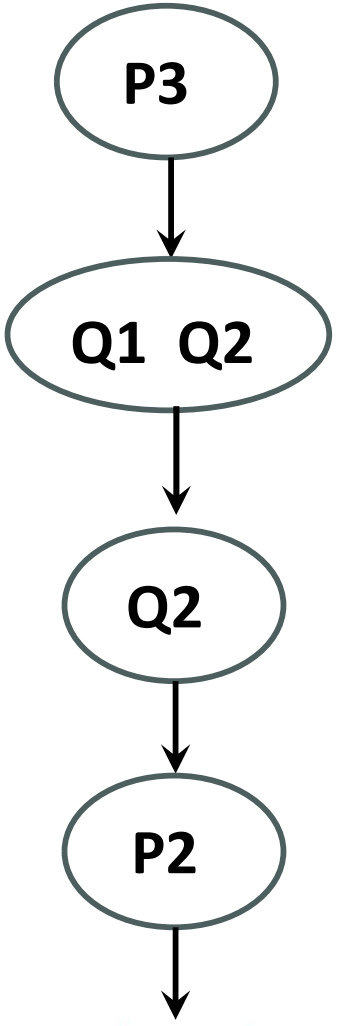
Base de Conocimiento (BC):

H1. P1 ($\rightarrow$ P1)
H2. P2 ($\rightarrow$ P2)
H3. Q1 ($\rightarrow$ Q1)
R1. Q1 $\wedge$ Q2 $\rightarrow$ P3
R2. P2 $\rightarrow$ Q2
R3. P1 $\wedge$ Q4 $\rightarrow$ Q3
R4. P1 $\wedge$ P3 $\rightarrow$ Q3

buscamos regla cuya cabeza unifique con el objetivo

el cuerpo de la regla con la que unifica son las nuevas cláusulas a satisfacer

Objetivo P3



unificación con cabeza de R1

Comenzamos por Q1. Unifica con H3.

unificación con cabeza de R2

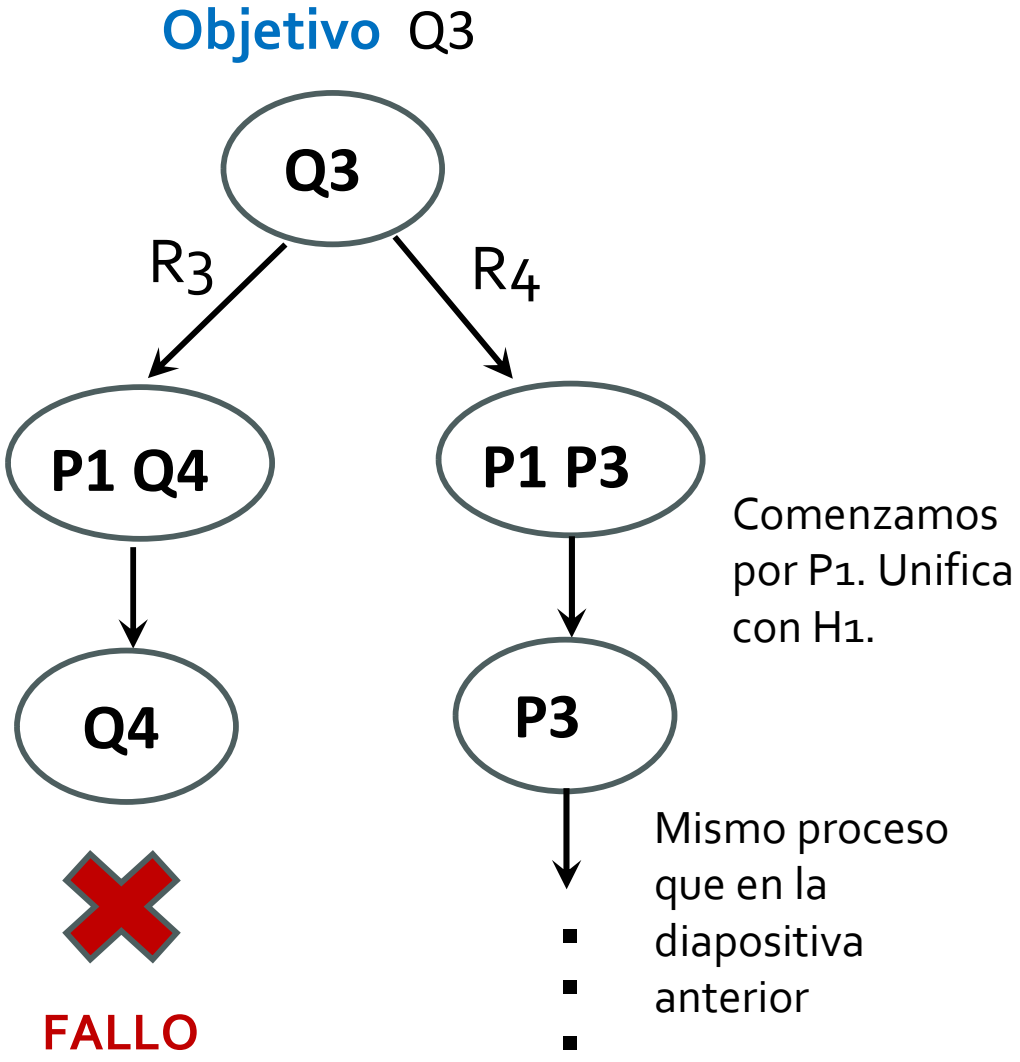
Unifica con H2



Programación lógica: resolución SLD

Base de Conocimiento (BC):

H1. P1 $(\rightarrow$ P1)
H2. P2 $(\rightarrow$ P2)
H3. Q1 $(\rightarrow$ Q1)
R1. Q1 $\wedge$ Q2 $\rightarrow$ P3
R2. P2 $\rightarrow$ Q2
R3. P1 $\wedge$ Q4 $\rightarrow$ Q3
R4. P1 $\wedge$ P3 $\rightarrow$ Q3



buscamos regla cuya cabeza unifique con el objetivo

Comenzamos por P1. Unifica con H1.

Comenzamos por P1. Unifica con H1.

Mismo proceso que en la diapositiva anterior



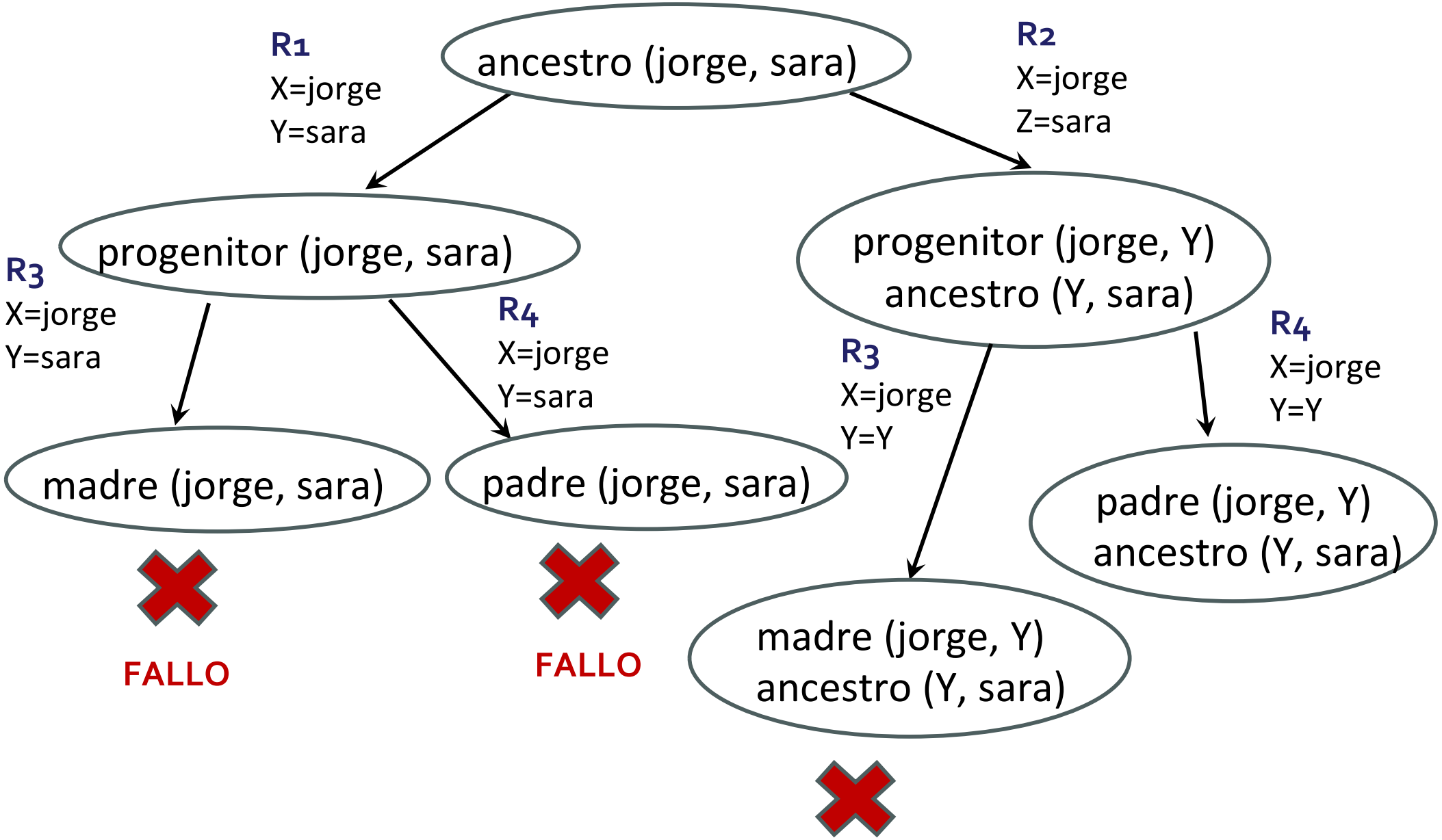
Ejemplo 3

Base de Conocimiento (BC):

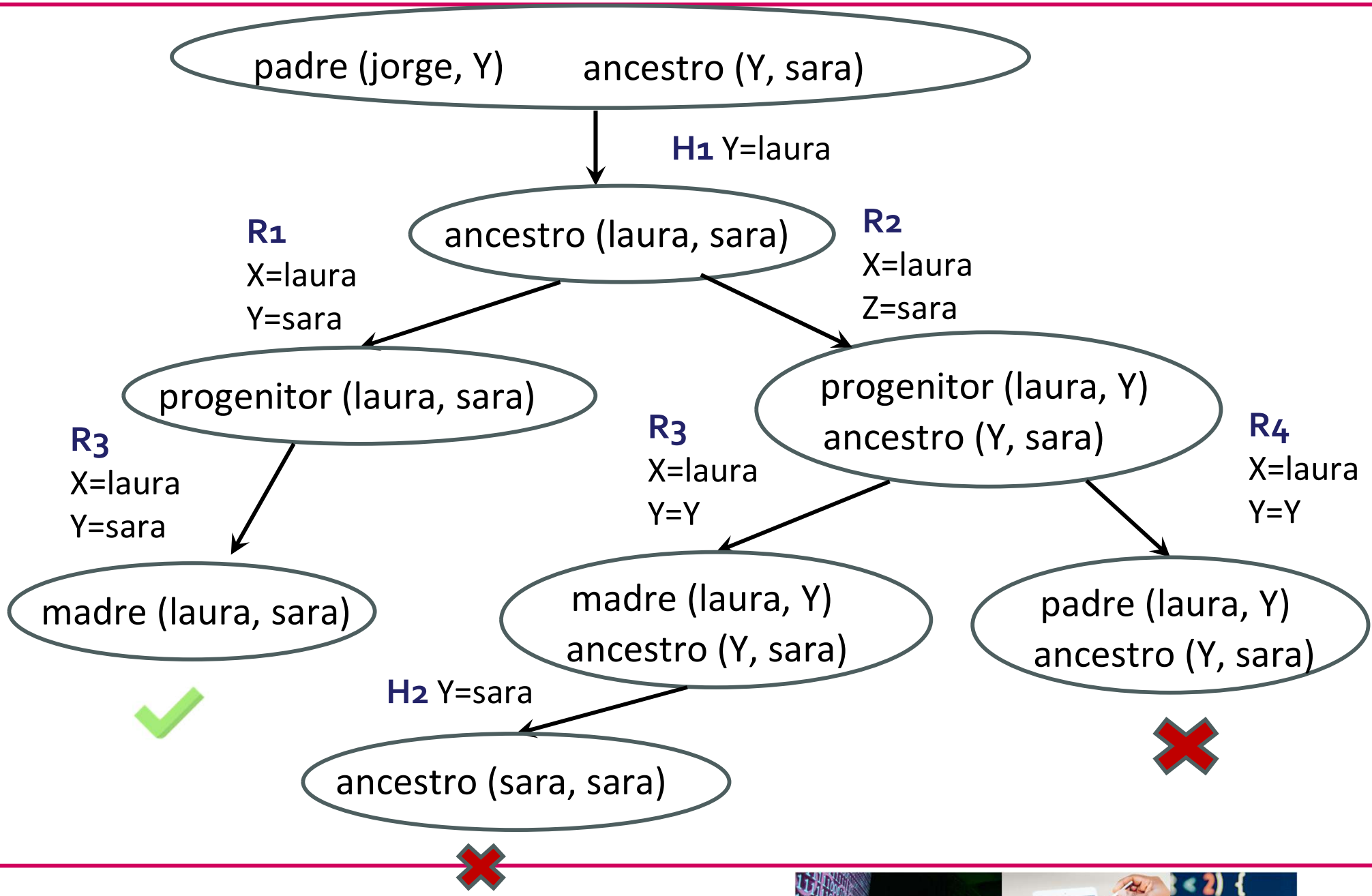
R1. progenitor (X,Y) $\rightarrow$ ancestro (X,Y)
R2. progenitor (X,Y) $\wedge$ ancestro (Y,Z) $\rightarrow$ ancestro (X,Z)
R3. madre (X,Y) $\rightarrow$ progenitor (X,Y)
R4. padre (X,Y) $\rightarrow$ progenitor (X,Y)
H1. padre (jorge, laura)
H2. madre (laura, sara)

Objetivo: ancestro (jorge, sara)?




ancestro (jorge, sara)
R1
X=jorge
Y=sara
R2
X=jorge
Z=sara
progenitor (jorge, sara)
progenitor (jorge, Y)
ancestro (Y, sara)
R3
X=jorge
Y=sara
R4
X=jorge
Y=sara
R3
X=jorge
Y=Y
R4
X=jorge
Y=Y
madre (jorge, sara)
padre (jorge, sara)
padre (jorge, Y)
ancestro (Y, sara)
madre (jorge, Y)
ancestro (Y, sara)
FALLO
FALLO




padre (jorge, Y) ancestro (Y, sara)
H1 Y=laura
ancestro (laura, sara)
R1
X=laura
Y=sara
R2
X=laura
Z=sara
progenitor (laura, sara)
progenitor (laura, Y)
ancestro (Y, sara)
R3
X=laura
Y=sara
R3
X=laura
Y=Y
R4
X=laura
Y=Y
madre (laura, sara)
madre (laura, Y)
ancestro (Y, sara)
padre (laura, Y)
ancestro (Y, sara)
H2 Y=sara
ancestro (sara, sara)



Programación lógica: resolución SLD

Combinando **unificación** y **encadenamiento hacia atrás** podemos obtener un procedimiento que nos permita responder a preguntas del tipo ¿se puede obtener P a partir de BC? ¿se puede satisfacer P a partir de BC?

Se trata de encontrar una secuencia de sustituciones/unificaciones tal que aplicadas a partir del objetivo P lleguemos a un objetivo que sea TRUE (ÉXITO).



Programación lógica: resolución SLD

Árbol SLD:

Los **nodos** de un árbol representan los objetivos pendientes (fórmulas por deducir)

Nodo raíz: objetivo inicial

Nodos finales:

- Un **nodo de fallo** se obtiene cuando una cláusula no unifica con la cabeza de ninguna regla o hecho
- Un **nodo de éxito** es aquel que no tiene objetivos pendientes

Estrategia de búsqueda:

- En nuestro caso, aplicamos **búsqueda en profundidad**
- Selección de objetivo a resolver: cualquier función de selección serviría
- También influye el orden en el que se usan las reglas y los hechos de la BC

Problema del algoritmo de resolución SLD: podría no encontrar una rama de éxito debido a la **existencia de ramas infinitas**



Ejercicio de resolución SLD

Base de Conocimiento (BC):

H1. matriculado(ana,logica)
H2. imparte(carlos,logica)
H3. departamento(carlos,ia)

R1. matriculado(X,A) ∧ imparte(P,A) → alumno_de(X,P)

R2. alumno_de(X,P) ∧ departamento(P,D) → pertenece_area(X,D)

Objetivo: pertenece_area(ana,ia)?



Corrección de SLD

Si SLD obtiene una respuesta mediante una refutación exitosa, entonces esa respuesta es una consecuencia lógica del programa.

Todo resultado producido por SLD es correcto →
no genera soluciones falsas

No implica que sea cierto en el mundo real, sino que es cierto según los hechos y reglas contenidos en la base de conocimiento.

Por tanto, SLD es **correcto**.



Completitud de SLD

Si una consulta es consecuencia lógica del programa y existe una derivación adecuada, SLD puede encontrarla.

Si una respuesta es verdadera según el programa lógico, existe una derivación SLD que la obtiene.

Por ello se dice que SLD es **completo** para programas de cláusulas definidas.



Decidibilidad de SLD

Un sistema lógico decidible es aquel para el que existe un algoritmo (un procedimiento mecánico, una serie de pasos bien definidos) que puede determinar, en un número finito de pasos, si cualquier fórmula dada en el lenguaje del sistema es una verdad lógica dentro de ese sistema o no.

SLD no es decidible:

- Si encuentra una prueba → responde.
- Si encuentra que no hay prueba en un espacio finito → responde.
- En algunos casos → no termina nunca:
  - Puede ocurrir que una derivación entre en un bucle infinito y nunca termine



Ventajas y limitaciones de la representación lógica



- Semántica formal bien definida.
- Inferencias automáticas.
- Base de numerosos sistemas basados en el conocimiento.
- Razonamiento formal y verificable.



- Excepciones y razonamiento por defecto.
- Información incompleta.
- Incertidumbre y vaguedad.
- Conocimiento temporal.
- Creencias y conocimiento de agentes.

Estas limitaciones han dado lugar al desarrollo de otros formalismos de representación del conocimiento, como las ontologías, los sistemas de producción y la lógica difusa, que estudiaremos en las siguientes unidades.



Ejercicio 1

R1. Si una persona bondadosa tiene afecto por otra que es rica entonces heredará de ella.
R2. Si una persona es amiga de otra entonces tiene afecto por ella.
R3. Si una persona es progenitor/a de otra entonces es su ancestro.
R4. Si una persona es ancestro de otra entonces esta tiene afecto por su ancestro.
R5. Si una persona es progenitor/a de otra y esta última es progenitor/a de una tercera persona entonces la primera persona es ancestro de la tercera.
H1. María es progenitora de Luis.
H2. Luis es progenitor de Lucía.
H3. Pedro es una persona rica.
H4. María es una persona rica.
H5. Lucía y Pedro son amigos.
H6. Lucía es una persona bondadosa.

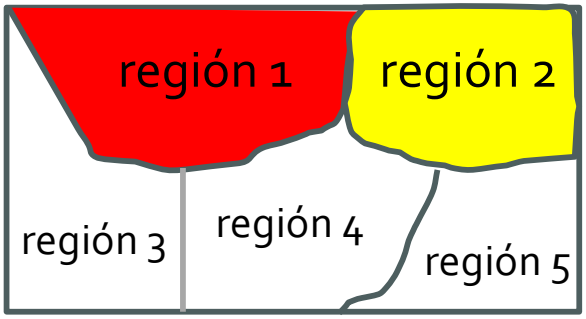
Objetivo: ¿Heredará Lucía de María?

1) Realiza la representación del conocimiento de este problema usando lógica de primer orden
2) Demuestra mediante resolución SLD si el objetivo se satisface o no



Ejercicio 2

Se dispone de una superficie dividida en regiones donde algunas de las regiones pueden estar pintadas de un color. Según las restricciones del problema, si una región está pintada de un color entonces ninguna región fronteriza puede estar pintada del mismo color.



Sabiendo que tenemos una superficie dividida en 5 regiones como se muestra en la figura, donde la región 1 está pintada de rojo, y la región 2 está pintada de amarillo, se pide realizar la representación del conocimiento de este problema usando la lógica de predicados de primer orden y demostrar mediante resolución SLD si se satisfacen o no lo siguientes objetivos:

(O1) ¿Se puede pintar la región 3 de amarillo?
(O2) ¿Se puede pintar la región 5 de amarillo?



Ejercicio 3

4			
		3	
	1		
			2

El Sudoku es un juego de estrategia que se desarrolla en un tablero $n \times n$. El juego consiste en poner los números 1, 2, …, n en cada fila, en cada columna y en cada uno de los n bloques de dimensión $\sqrt{n} \times \sqrt{n}$ en los que se divide el tablero.

Es decir, cada fila, columna y bloque debe contener los números del 1 al n sin repetirse.

Dado el sudoku de la figura se pide realizar la representación del conocimiento de este problema usando la lógica de predicados de primer orden y demostrar mediante resolución SLD si se satisfacen o no lo siguientes objetivos:

1. ¿Se puede poner un 4 en la casilla (2,1)?
2. ¿Se puede poner un 1 en la casilla (2,1)?
3. ¿Se puede poner un 2 en la casilla (2,1)?
4. ¿Se puede poner un 3 en la casilla (2,1)?
5. ¿Se puede poner un 1 en la casilla (1,1)?

NOTA: En este problema se admite utilizar operaciones aritméticas como en pyDatalog

